Laboratory 8
Multilayer perceptrons

1. Linear Regression in PyTorch

- construct an artificial dataset, $y = Xw + b + \epsilon$;

```python
def synthetic_data(w, b, num_examples):
    """Generate y = Xw + b + noise."""
    X = torch.normal(0, 1, (num_examples, len(w)))
    y = X @ w + b
    y += torch.normal(0, 0.01, y.shape)
    return X, y.reshape(-1, 1)

true_w = torch.tensor([2, -3.4])
true_b = 4.2
features, labels = synthetic_data(true_w, true_b, 1000)
```

  - $y = 2x_1 - 3.4x_2 + 4.2 + \epsilon$;
  - $X \in \mathbb{R}^{1000 \times 2}$

- training ML models consists of making multiple passes over the dataset, fetching one mini-batch of examples at a time, and using them to update the model;

```python
def load_array(data_arrays, batch_size, is_train=True):
    """Construct a PyTorch data iterator."""
    dataset = torch.utils.data.TensorDataset(*data_arrays)
    return torch.utils.data.DataLoader(dataset, batch_size, shuffle=is_train)

batch_size = 10
data_iter = load_array((features, labels), batch_size)
```

  - shuffle the dataset and access it in mini-batches;

1. Linear Regression in PyTorch

- define the linear regression model, initialize the parameters, specify the loss function and the optimizer to be used;

```
net = nn.Sequential(nn.Linear(2, 1))

net[0].weight.data.normal_(0, 0.01)
net[0].bias.data.fill_(0)

loss = nn.MSELoss()

optimizer = torch.optim.SGD(net.parameters(), lr=0.03)
```

- *nn.MSELoss* class computes the mean squared error (without $\frac{1}{2}$ factor);
- it returns the average loss over examples;

1. Linear Regression in PyTorch

- Steps for training the linear regression model:
  1. generate predictions (*net(X)*) and compute the loss (forward propagation);
  2. calculate gradients by running the backpropagation, using the *backward()* function on the loss;
  3. update the model parameters using *optimizer.step()*;

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}).$$

We compute the loss after each epoch and print it to monitor progress.

```
num_epochs = 3
for epoch in range(num_epochs):
    for X, y in data_iter:
        l = loss(net(X), y)
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
    l = loss(net(features), labels)
    print(f'Epoch {epoch + 1}, Loss {l:f}')
```

2. Softmax Regression in PyTorch

- dataset: Fashion-MNIST (28×28 images, 10 classes, 60000 training images, 10000 test images)

```
def load_data_fashion_mnist(batch_size, resize=None):
    """Download the Fashion-MNIST dataset and then load it into memory."""
    ...
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [50000, 10000],
                                                           generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True,
                                        num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False,
                                        num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False,
                                        num_workers=2))

batch_size = 256
train_iter, val_iter, test_iter = load_data_fashion_mnist(batch_size)
```

- train set: 50000;
- validation set: 10000;

- define the softmax regression model;

```
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))

def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

net.apply(init_weights)

loss = nn.CrossEntropyLoss()

optimizer = torch.optim.SGD(net.parameters(), lr=0.1)
```

- $784 = 28 \times 28$;
- 10 classes $\Rightarrow$ output dimension of 10;

2. Softmax Regression in PyTorch

- the training process employs the following methods: *evaluate_accuracy*, *train_epoch* and *train*;
- *evaluate_accuracy*:

```
def evaluate_accuracy(net, data_iter):
    """Compute the accuracy for a model on a dataset."""
    net.eval()
    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

- *total_hits* – no. of correct predictions;

- *train_epoch*:

```
def train_epoch(net, train_iter, loss, optimizer):
    net.train()
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```

- train the model for one epoch;

2. Softmax Regression in PyTorch

- *train*:

```
def train(net, train_iter, val_iter, loss, num_epochs, optimizer):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f},
                  Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')

    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```

- train the model for multiple epochs, which is specified by *num_epochs*;

- train the model for 10 epochs:

```
num_epochs = 10
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, loss, num_epochs, optimizer)
```

3. Multilayer perceptrons

- **multilayer perceptron** (**MLP**) is a fully-connected multilayer neural network (Figure 1);
- the outputs $\mathbf{O} \in \mathbb{R}^{n \times q}$ of the one-hidden-layer MLP are computed as follows:

$$\mathbf{H} = \sigma(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}),$$
$$\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)},$$

where,

1. $\mathbf{X} \in \mathbb{R}^{n \times d}$ is the matrix of n examples, where each example has d features;
2. $\mathbf{H} \in \mathbb{R}^{n \times h}$ represents the outputs of the hidden layer;
3. $\sigma(\cdot)$ is the activation function;
4. $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times h}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times h}$ are the hidden-layer weights and biases;
5. $\mathbf{W}^{(2)} \in \mathbb{R}^{h \times q}$, $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times q}$ are the output-layer weights and biases.

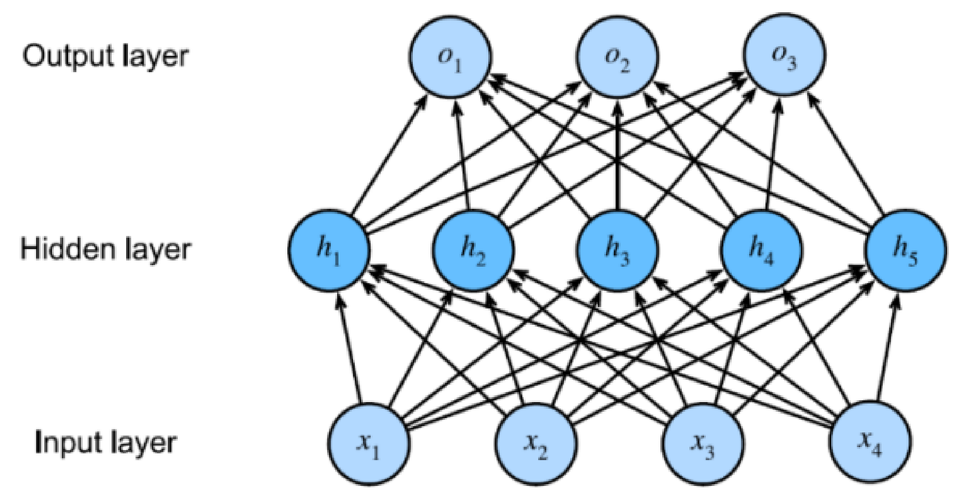


Figure 1: MLP with one hidden layer of 5 hidden units/neurons.

3. Multilayer perceptrons

- conventionally, we do not consider the input layer when counting layers, $L = 2$ layers in Figure 1;
- common activation functions:
  1. *rectified linear unit* (ReLU);
  $$ReLU(x) = max(x, 0).$$
  2. *parameterized ReLU* (pReLU);
  $$pReLU(x) = max(0, x) + \alpha min(0, x).$$
  3. *sigmoid*;
  $$sigmoid(x) = \frac{1}{1 + e^{-x}} \in (0, 1).$$
  4. *hyperbolic tangent* (tanh);
  $$tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}.$$

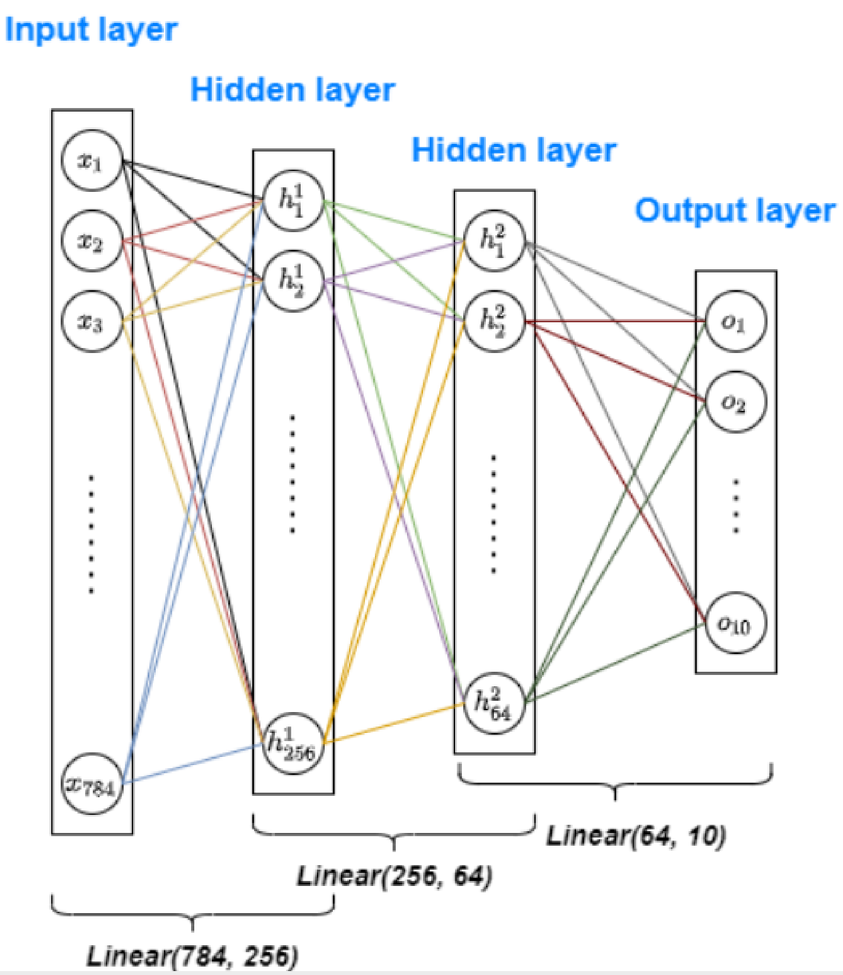
3.1 MLPs in PyTorch

```
net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    nn.Linear(256, 64),
                    nn.ReLU(),
                    nn.Linear(64, 10))

print(net)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): ReLU()
  (3): Linear(in_features=256, out_features=64, bias=True)
  (4): ReLU()
  (5): Linear(in_features=64, out_features=10, bias=True)
)
```

- MLP with two hidden layers;
- ReLU activation function is employed;



4. Model selection, overfitting, and underfitting

- the goal of machine learning algorithms is to **discover patterns** in the training dataset (images, text, sounds, etc.);
- the fundamental problem of machine learning is to discover patterns that **generalize** (i.e., successfully assess the performance for never encountered samples (test data));
- the phenomenon of fitting the training data more closely than we fit the underlying distribution is **overfitting**, and the techniques used to prevent overfitting are **regularization techniques**;
- **model selection** is the process of selecting the final model after evaluating several candidate models;
- a *validation dataset* is employed for selecting the final model;

4. Model selection, overfitting, and underfitting

- for example, with MLPs we wish to compare models with different number of hidden layers, different number of hidden units, various choices of activation functions;
- if the generalization gap between our training and validation errors is small, we have reason to believe that we could use a more complex model;
- this phenomenon is known as **underfitting**.

5. Weight decay

- **weight decay** (ℓ_2 regularization) technique replaces the original objective, minimizing the prediction loss on the training labels, with the new objective, minimizing the sum of the prediction loss and the penalty term;
- for example, consider the MSE loss given by:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2.$$

- for the one-hidden-layer MLP, Figure 1, the new objective is:

$$\mathcal{L}(\mathbf{w}) + \frac{\lambda}{2}(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2),$$

where the Frobenius norm is defined as: $\|\mathbf{W}\|_F = \sqrt{Tr(\mathbf{W}^T\mathbf{W})}$.

6. Dropout

- **dropout** technique refers to the practice of dropping out certain nodes/neurons during training;
- throughout training, on each iteration, standard dropout consists of zeroing out some fraction of the nodes in each layer, before calculating the subsequent layer;
- for example, when we apply dropout to the hidden layer of MLP architecture in Figure 1, zeroing out each hidden unit with probability p, the result can be viewed as a network containing only a subset of the original neurons, Figure 2.

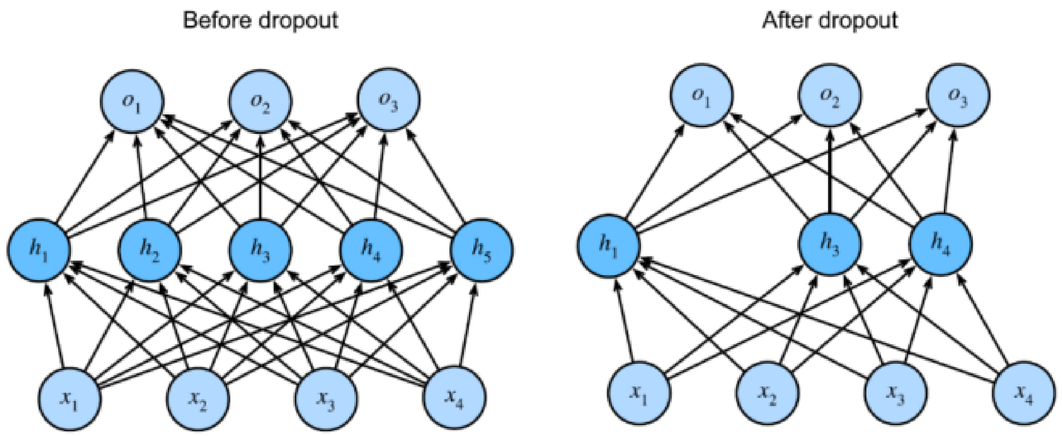


Figure 2: MLP before and after dropout.

7. Weight decay and Dropout in PyTorch

```
wd = 0.001
optimizer = torch.optim.SGD(net.parameters(), weight_decay=wd, lr=lr)
```

```
dropout1, dropout2 = 0.2, 0.3

net = nn.Sequential(nn.Flatten(),
        nn.Linear(784, 256),
        nn.ReLU(),
        # Add a dropout layer after the first fully connected layer
        nn.Dropout(dropout1),
        nn.Linear(256, 256),
        nn.ReLU(),
        # Add a dropout layer after the second fully connected layer
        nn.Dropout(dropout2),
        nn.Linear(256, 10))
```

- PyTorch integrates weight decay into the optimization algorithm;
- *nn.Dropout* layer;
- drop 20% of the outputs of the first hidden layer and 30% of the outputs of the second hidden layer;

8. Deep learning computation

- in order to implement complex neural networks, the *block* concept is introduced;
- a block could describe a single layer, a component consisting of multiple layers, or the entire model itself;
- from a software standpoint, a *block* is a *class*;

```
class MLP(nn.Module):

    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(20, 256)  # Hidden layer
        self.out = nn.Linear(256, 10)  # Output layer

    def forward(self, X):
        return self.out(nn.ReLU()(self.hidden(X)))
```

- one-hidden-layer MLP;
- the forward propagation function returns the required model output, based on the input X;

8. Deep learning computation

```
net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 1))
print(net)

#print(net[0].state_dict()) # parameters of the first fully-connected layer
print(net[2].state_dict())  # parameters of the second fully-connected layer
```

```
Sequential(
  (0): Linear(in_features=4, out_features=8, bias=True)
  (1): ReLU()
  (2): Linear(in_features=8, out_features=1, bias=True)
)
OrderedDict([('weight', tensor([[-0.2609,  0.0844,  0.0202, -0.2051, -0.2642,  0.1080, -0.2782, -0.1413]])), ('bias', tensor([0.3130]))])
```

➢ inspect the learnable parameters of a neural network;

```
def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu()."""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')

net = nn.Sequential(nn.Linear(3, 1))
net = net.to(device=try_gpu())
```

➢ model parameters are placed on the GPU in order to accelerate the training process;